

K L Deemed to be University

Department of Computer Science and Engineering | 2026–2027, Odd Semester

Database Systems Engineering and Distributed Backend Development (25CS1302E)

Practical Experiment 2: The Librarian's Dashboard

Name	Chakrika Panakala
Roll No.	2520090042
Course	25CS1302E
CO Mapping	CO1
Session	Practical 2
Date	_____

Aim

To create the Loans junction table and report each member with the book they have currently borrowed (JOIN), count books per published year (GROUP BY), implement the book-donation process as an atomic transaction with a Donation_History log, and create an index on ISBN and explain the speed difference.

Program

File: p2.sql | Prerequisite: Experiment 1

```
-- Practical Experiment 2: The Librarian's Dashboard
-- Student: Chakrika Panakala | Roll No.: 2520090042
-- Prerequisite: bookflow_db from Experiment 1

\c bookflow_db

-- Additional books for the dashboard reports
INSERT INTO Books (title, isbn, published_year) VALUES
('Dune', '9780441172719', 2019),
('Atomic Habits', '9780735211292', 2018),
('Designing Data-Intensive Applications', '9781449373320', 2017),
('Project Hail Mary', '9780593135204', 2021),
('The Pragmatic Programmer', '9780135957059', 2019),
('Deep Work', '9781455586691', 2016);

-- Additional members
INSERT INTO Members (full_name, email) VALUES
('Meera Shah', 'meera.shah@outlook.com'),
('Arjun Nair', 'arjun.nair@gmail.com');

-- Task 1: Loans junction table
CREATE TABLE Loans (
    loan_id SERIAL PRIMARY KEY,
    member_id INT NOT NULL REFERENCES Members(member_id) ON DELETE CASCADE,
    book_id INT NOT NULL REFERENCES Books(book_id) ON DELETE CASCADE,
    loan_date DATE NOT NULL DEFAULT CURRENT_DATE,
    return_date DATE,
    CHECK (return_date IS NULL OR return_date >= loan_date)
);

-- Insert loans using email and ISBN rather than generated IDs
INSERT INTO Loans (member_id, book_id, loan_date, return_date)
SELECT m.member_id, b.book_id, v.loan_date::DATE, v.return_date::DATE
FROM (VALUES
    ('chakrika.panakala@gmail.com', '9780553418026', '2026-09-20', NULL),
    ('chakrika.panakala@gmail.com', '9780441172719', '2026-09-25', NULL),
    ('meera.shah@outlook.com', '9780132350884', '2026-09-10', '2026-09-24'),
    ('meera.shah@outlook.com', '9781449373320', '2026-09-28', NULL),
    ('arjun.nair@gmail.com', '9780593135204', '2026-09-15', NULL),
    ('arjun.nair@gmail.com', '9780078022159', '2026-09-01', '2026-09-12')
) AS v(email, isbn, loan_date, return_date)
JOIN Members m ON m.email = v.email
JOIN Books b ON b.isbn = v.isbn;

SELECT * FROM Loans ORDER BY loan_id;
```

```

-- Currently borrowed books
SELECT m.full_name AS member_name,
       b.title AS book_title,
       l.loan_date
FROM Loans l
JOIN Members m ON m.member_id = l.member_id
JOIN Books b ON b.book_id = l.book_id
WHERE l.return_date IS NULL
ORDER BY m.full_name, l.loan_date;

-- Task 2: Books published in each year
SELECT published_year,
       COUNT(book_id) AS total_books
FROM Books
GROUP BY published_year
ORDER BY published_year;

-- Task 3: Donation history and atomic transaction
CREATE TABLE Donation_History (
    donation_id SERIAL PRIMARY KEY,
    book_id INT NOT NULL REFERENCES Books(book_id),
    donor_name VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL,
    donated_on TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Case A: successful transaction
BEGIN;
INSERT INTO Books (title, isbn, published_year)
VALUES ('Introduction to Algorithms', '9780262046305', 2022);

INSERT INTO Donation_History (book_id, donor_name, category)
VALUES (currval('books_book_id_seq'), 'Rotary Club Vijayawada', 'Computer Science');
COMMIT;

-- Case B: failed second step, so the whole transaction is rolled back
BEGIN;
INSERT INTO Books (title, isbn, published_year)
VALUES ('Sapiens', '9780062316097', 2015);

INSERT INTO Donation_History (book_id, donor_name, category)
VALUES (currval('books_book_id_seq'), 'Anonymous', NULL);
ROLLBACK;

SELECT b.book_id, b.title, d.donor_name, d.category
FROM Donation_History d
JOIN Books b ON b.book_id = d.book_id;

SELECT COUNT(*) AS sapiens_rows
FROM Books
WHERE title = 'Sapiens';

-- Task 4: ISBN indexing
CREATE INDEX idx_books_isbn ON Books (isbn);
\di idx_books_isbn

-- Larger copy for performance comparison
CREATE TABLE books_large AS
SELECT g AS book_id,
       'Book Title ' || g AS title,
       (9780000000000 + g)::TEXT AS isbn,
       1950 + (g % 76) AS published_year
FROM generate_series(1, 50000) AS g;

ANALYZE books_large;

-- Before indexing
EXPLAIN ANALYZE
SELECT * FROM books_large
WHERE isbn = '9780000042424';

CREATE INDEX idx_books_large_isbn ON books_large (isbn);
ANALYZE books_large;

-- After indexing
EXPLAIN ANALYZE
SELECT * FROM books_large
WHERE isbn = '9780000042424';

-- Unindexed title search
EXPLAIN ANALYZE
SELECT * FROM books_large
WHERE title = 'Book Title 42424';

```

Output

The database was first extended with additional books and members from the previous practical. The following outputs show the four required tasks: JOIN-based loan reporting, GROUP BY statistics, transaction handling, and ISBN indexing.

1. Loans Table

loan_id	member_id	book_id	loan_date	return_date
1	1	3	2026-09-20	—
2	1	4	2026-09-25	—
3	4	1	2026-09-10	2026-09-24
4	4	7	2026-09-28	—
5	5	9	2026-09-15	—
6	5	2	2026-09-01	2026-09-12

2. Currently Borrowed Books (JOIN)

Member Name	Book Title	Loan Date
Arjun Nair	Project Hail Mary	2026-09-15
Chakrika Panakala	The Martian	2026-09-20
Chakrika Panakala	Dune	2026-09-25
Meera Shah	Designing Data-Intensive Applications	2026-09-28

3. Books Published by Year (GROUP BY)

Published Year	Total Books
2008	1
2014	1
2016	1
2017	1
2018	1
2019	3
2021	1

4. Donation Transaction

Case A – Commit: The book “Introduction to Algorithms” and its corresponding donation history entry were inserted successfully. The transaction completed with COMMIT.

Case B – Rollback: The donation record was given a NULL category. Since category is NOT NULL, the second INSERT failed. ROLLBACK then removed the earlier Sapiens insertion as well.

book_id	title	donor_name	category
13	Introduction to Algorithms	Rotary Club Vijayawada	Computer Science

Sapiens count after ROLLBACK: 0

5. ISBN Indexing and Speed Comparison

Search	Plan	Execution time (sample output)
ISBN before index	Sequential Scan	3.475 ms
ISBN after index	Index Scan (B-tree)	0.032 ms
Title without index	Sequential Scan	3.473 ms

Before the index, the ISBN query used a Sequential Scan and checked all 50,000 rows. After creating the B-tree index, PostgreSQL used an Index Scan and located the matching ISBN much faster. The title query remained a Sequential Scan because no title index was created.

Result

The Loans junction table was created and JOIN was used to display members with their currently borrowed books. GROUP BY successfully counted books according to publication year. The donation workflow showed transaction atomicity because the successful case committed both operations while the failed case was rolled back completely. Finally, the ISBN index changed the search plan from a sequential scan to a B-tree index scan and significantly reduced the sample execution time.

Conclusion

This practical demonstrated how relational database operations can support a librarian's dashboard. JOIN connects related records, GROUP BY produces collection statistics, transactions protect multi-step updates, and indexing improves lookup performance for frequently searched columns such as ISBN.